

Region Resonances

Technical Appendix

Mathematical Framework, Algorithms, and Implementation

Jure Skarabot · May 2026

Companion documents: Summary · Methods Primer · Data Appendix · Region Reference — Identity

This appendix presents the mathematical framework underlying the Region Resonances matching procedure: the embedding map that converts text to vectors, the geometry of the unit sphere on which all vectors live, the cosine-similarity score that ranks regions against a query, the display transformation that converts raw similarities into the percentages users see, and the implementation details of the deployed pipeline. Notation follows standard linear algebra; treatments are stated rather than derived, with references provided for full derivations. The companion *Methods Primer* provides a non-technical guide to the same material.

A. From Text to Vectors

A.1 The Embedding Map

The pipeline begins with a function that converts text into numerical vectors. Such a function is called an *embedding map*. Formally, if $\mathcal{S}$ denotes the set of finite text strings (passages of arbitrary length, including each user query and each region’s narrative), then an embedding map of dimension d is a function

$$E : \mathcal{S} \longrightarrow \mathbb{R}^d$$

that assigns to each text string s a real vector $E(s) \in \mathbb{R}^d$. In the deployed system, the embedding map is OpenAI’s `text-embedding-3-small` model, and the output dimension is $d = 1536$.

The embedding model is treated throughout as a fixed, opaque function: how its weights were learned and how its computation is structured are out of scope. What matters for the matching procedure is the geometric structure that the map induces on $\mathbb{R}^d$.

A.2 Unit Normalisation

The OpenAI map has a useful property: every output vector is rescaled to have Euclidean length 1. That is, for every input string s ,

$$\|E(s)\|_2 = 1, \quad \text{where } \|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_d^2}.$$

The image of the embedding map therefore lies entirely on the surface of the unit sphere in $\mathbb{R}^d$, denoted

$$S^{d-1} = \{\mathbf{v} \in \mathbb{R}^d : \|\mathbf{v}\|_2 = 1\}.$$

Every text the system processes — each of the 59 region passages, every user query — is represented as a point on this sphere. The matching procedure is therefore a question about the geometry of the unit sphere: given a query point and 59 region points, which region points are closest?

A.3 The Corpus

Let the corpus consist of $n = 59$ region passages $r_1, r_2, \dots, r_n$, where each r_i is the concatenation of a region’s one-word identity metaphor with its full Layer 1 identity narrative. The embedding map produces 59 vectors:

$$\mathbf{r}_i = E(r_i) \in S^{d-1}, \quad i = 1, \dots, n.$$

These vectors are computed once, in advance, and stored. They are not recomputed at runtime. The 59 vectors collectively form a fixed reference set against which every user query is compared. They are stacked as rows of a matrix $R \in \mathbb{R}^{n \times d}$.

B. Cosine Similarity

B.1 Definition

The standard way to measure similarity between two vectors on the unit sphere is cosine similarity, defined for two nonzero vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ as

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2},$$

where $\langle \mathbf{x}, \mathbf{y} \rangle = x_1 y_1 + x_2 y_2 + \dots + x_d y_d$ is the standard inner product. Geometrically this is the cosine of the angle between $\mathbf{x}$ and $\mathbf{y}$ regarded as arrows from the origin. It takes values in $[-1, +1]$: $+1$ when the vectors point in the same direction, 0 when they are perpendicular, -1 when they point in opposite directions.

B.2 The Unit-Sphere Simplification

Because all vectors in this system are unit-normalised, the denominator in the cosine-similarity formula is always $1 \cdot 1 = 1$. The formula collapses to

$$\cos(\mathbf{r}_i, \mathbf{q}) = \langle \mathbf{r}_i, \mathbf{q} \rangle, \quad \mathbf{r}_i, \mathbf{q} \in S^{d-1}.$$

Cosine similarity on the unit sphere is just the inner product. This is the central computational identity of the system: every similarity measurement is one inner-product calculation, with no division required.

B.3 Equivalence of Closeness Notions on the Sphere

On the unit sphere, three natural ways of measuring how close two points are happen to give the same ranking:

- Cosine similarity $\langle \mathbf{r}_i, \mathbf{q} \rangle$, ranging from -1 to $+1$, with higher values meaning more similar.
- Chord distance $\|\mathbf{r}_i - \mathbf{q}\|_2$, the straight-line distance through the interior of the sphere.
- Angular distance $\arccos\langle \mathbf{r}_i, \mathbf{q} \rangle$, the great-circle distance along the surface.

These three quantities are different numerical values, but they always agree on which of two candidates is closer to a fixed query: ranking the corpus by descending cosine similarity is identical to ranking it by ascending chord distance, which is identical to ranking it by ascending angular distance. Cosine similarity is preferred because it is bounded in $[-1, +1]$, dimensionless, and naturally interpretable as angular alignment.

C. The Matching Procedure

C.1 Computing the Similarity Vector

When the user submits a query q , the embedding map produces the query vector

$$\mathbf{q} = E(q) \in S^{d-1}.$$

For each region $i = 1, \dots, n$, the system computes the cosine similarity $s_i = \langle \mathbf{r}_i, \mathbf{q} \rangle$. The 59 similarities can be collected into a vector $\mathbf{s} = (s_1, s_2, \dots, s_n) \in \mathbb{R}^n$. Stacking the region vectors as rows of R , the entire similarity vector is one matrix–vector product:

$$\mathbf{s} = R \mathbf{q}.$$

C.2 Top- k Selection

Given the similarity vector $\mathbf{s}$, the system sorts the indices in descending order of similarity and returns the top k :

$$T_k(q) = \text{argtop-}k_{i \in \{1, \dots, n\}} s_i.$$

In deployment, $k = 5$, so the user sees the five regions with the highest cosine similarity to the query. The remaining 54 are accessible through a disclosure control. Geometrically, this procedure is a nearest-neighbour search on S^{d-1} .

D. The Display Score

D.1 The Empirical Range of Cosine Similarity

Although cosine similarity is mathematically bounded in $[-1, +1]$, the values actually observed on this corpus occupy a much narrower interval. Across a calibration test set of 60 queries spanning words, phrases, and short prose fragments, the observed similarities s_i fell almost entirely within $[0.15, 0.65]$. Values below 0.15 occur only for pathological inputs (random byte sequences, unrelated languages); values above 0.70 occur only when the query is itself a paraphrase of source-corpus text.

This concentration is a generic feature of high-dimensional embeddings of natural-language text. Almost any two passages of English share a substantial baseline of common semantic structure — they are both grammatical English, share vocabulary, refer to a shared world — and the embedding model encodes this shared structure into a common region of the sphere. The signal that distinguishes one passage from another lives in a relatively narrow range above the baseline.

D.2 Why Raw Cosine Is Not Displayed

Presenting raw cosine values to users would be uninformative. A score of 0.62 sounds like a 62% result but in fact represents a strong match; a score of 0.31 sounds like failure but in fact represents a typical similarity for an unrelated query. Worse, the differences that matter most — the spread between the top match and the tenth match — often occupy only a few hundredths of the cosine range, where they would be visually invisible.

D.3 Clip-and-Rescale

To present scores in a range that uses the full visual scale, a piecewise-linear transformation is applied. Define constants $s_{\min} = 0.15$ and $s_{\max} = 0.70$, calibrated against the 60-query test set so that the operating range of the cosine corresponds to the operating range of the display. The display score is then

$$\tilde{s}_i = \text{clip}\left(\frac{s_i - s_{\min}}{s_{\max} - s_{\min}}, 0, 1\right) \times 100,$$

where $\text{clip}(t, 0, 1) = \min(1, \max(0, t))$. The transformation maps the empirical operating range $[0.15, 0.70]$ linearly onto $[0, 100]$ and clips anything outside that range to the boundary.

This transformation has three properties that matter for interpretation:

- It is bounded between 0 and 100, so displayed scores always fit on a percentage scale.
- It is monotonically increasing on the operating range, so the displayed ranking is always identical to the underlying cosine ranking.
- It is affine on the operating range, so a difference of 10 in the displayed score corresponds to a fixed difference in the underlying cosine, regardless of where on the scale that difference falls.

Two regions whose displayed scores differ by 10 are equally distinguishable in the underlying space whether the scores are 40 and 50 or 80 and 90.

D.4 Why Not Softmax

A common alternative for displaying similarity scores as percentages is the softmax transformation, which produces a probability distribution over the corpus:

$$\pi_i = \frac{\exp(\beta s_i)}{\sum_{j=1}^n \exp(\beta s_j)}, \quad \beta > 0.$$

This also preserves the ranking, but it has two drawbacks for the present application. First, the choice of the temperature parameter β is arbitrary, and different values produce dramatically different displayed probabilities for the same underlying ranking; there is no principled way to set it. Second, softmax is multiplicative in the score gap: the ratio π_i/π_j grows exponentially in $s_i - s_j$. A very small gap in cosine similarity becomes a very large gap in displayed probability, exaggerating differences between near-tied results and giving users a misleading sense of confidence in the top match.

Clip-and-rescale, by contrast, treats equal cosine differences as equal display differences. Two regions that are nearly tied in true semantic similarity are displayed as nearly tied. This is the honest representation of what the matching procedure has found.

E. Algorithms

Algorithm 1 Pre-computation of corpus embeddings (offline, run once)

Require: Region passages $\{r_i\}_{i=1}^{n=59}$, each a concatenation of metaphor and identity narrative

- 1: $R \leftarrow$ empty $n \times d$ matrix, $d = 1536$
- 2: **for** $i = 1$ to n **do**
- 3: $\mathbf{r}_i \leftarrow E(r_i)$ $\triangleright$ call OpenAI `text-embedding-3-small`
- 4: assert $\|\mathbf{r}_i\|_2 = 1$ $\triangleright$ model returns unit-normalised vectors
- 5: $R_{i,:} \leftarrow \mathbf{r}_i$
- 6: **end for**
- 7: serialise R to JSON, ship with the application

Algorithm 2 Query-time matching (online, per query)

Require: Query string q , corpus matrix $R \in \mathbb{R}^{n \times d}$, top- k parameter (default 5)

- 1: $\mathbf{q} \leftarrow E(q)$ $\triangleright$ embed the query with the same model
- 2: $\mathbf{s} \leftarrow R\mathbf{q}$ $\triangleright$ single matrix-vector product, $O(nd)$
- 3: $\mathcal{T} \leftarrow$ indices of the top k entries of $\mathbf{s}$
- 4: **for** $i \in \mathcal{T}$ **do**
- 5: $\tilde{s}_i \leftarrow \text{clip}((s_i - 0.15)/(0.70 - 0.15), 0, 1) \times 100$
- 6: **end for**
- 7: **return** ordered list of $(i, s_i, \tilde{s}_i)$ for $i \in \mathcal{T}$

F. Implementation

F.1 Technology Stack

The corpus pre-computation pipeline is implemented in Python, calling the OpenAI Embeddings API and writing results to a static JSON file. The web tool is a single-page React application built with `vite`; the matching procedure runs in the browser using `tensorflow.js` for the matrix-vector product, with the corpus matrix loaded from the static JSON. Query embeddings are computed by calling the OpenAI Embeddings API from a small server-side proxy that holds the API key.

F.2 Computational Pipeline

1. *Corpus assembly.* The 59 region passages are constructed as the concatenation of metaphor and identity narrative, in the form “`{metaphor}. {narrative}`”.
2. *Corpus embedding.* Each passage is embedded by `text-embedding-3-small`; the 59 unit-norm vectors are stacked into the matrix R and serialised.
3. *Query embedding.* At runtime, the user’s query is sent to the OpenAI Embeddings API via the proxy, which returns the unit-norm query vector.
4. *Similarity computation.* The browser computes $\mathbf{s} = R\mathbf{q}$.
5. *Top- k and display.* The top five indices are selected, the clip-and-rescale transformation is applied, and the result is rendered.

F.3 Computational Cost

Embedding cost is one API call per query, with negligible compute on the client. The matrix–vector product $R\mathbf{q}$ requires $nd = 59 \cdot 1536 \approx 9 \times 10^4$ multiply-add operations, which is microseconds on any modern device. Storage cost for the corpus matrix is $nd \cdot 4 \approx 360$ KB at single-precision float, comfortable for a static asset.

F.4 Privacy

Queries are transmitted to the embedding service for the purpose of computing the semantic embedding. They are not stored, logged, or aggregated by the Region Resonances application. There are no user accounts and no session-spanning history. The same query submitted twice produces the same result, but this is a property of the model and the corpus, not of any record kept of the user’s previous activity.

F.5 What the Mathematics Does and Does Not Establish

The procedure described above is a sequence of well-defined operations: produce a vector via the embedding map, compute inner products against a fixed reference set, sort, transform for display. Each step is mathematically clean and computationally cheap. Given the same query, the procedure always returns the same ranking, and small changes to the query produce small changes in the similarity vector — both desirable properties.

What the mathematics does not establish is whether the rankings correspond to human judgments of semantic similarity. That property depends entirely on the embedding map E : whether it has learned, from the text it was trained on, to assign nearby vectors to passages that competent readers would judge similar in meaning. This is an empirical question about the embedding model, validated on standard benchmarks but not formally proven. The mathematical framework is therefore a guarantee of well-posedness, not of meaningfulness.

References

- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26.
- Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proc. EMNLP-IJCNLP 2019*, 3982–3992.
- OpenAI (2024). *New embedding models and API updates*. `text-embedding-3-small` model documentation. OpenAI Platform.
- Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Salton, G., Wong, A. & Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613–620.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Section 4.3 (softmax) and Section 12.1 (PCA on the sphere).